

Rajarshi Shahu College of Engineering, Tathwade Pune-33

(An Empowered Autonomous Institute Affiliated to Savitribai Phule Pune University
Pune)

ASSISGNMENT NO 4

Building a Custom Docker Image Using a Dockerfile and Managing Images in a Container Registry

AIM

To build a custom Docker image using a Dockerfile and manage the image by tagging, pushing, and pulling it from a container registry.

OBJECTIVES

1. To understand the concept and structure of a Dockerfile.
2. To create a custom Docker image using a Dockerfile.
3. To build and test a Docker image locally.
4. To tag a Docker image for use with a container registry.
5. To push and pull Docker images from a container registry.
6. To understand basic Docker image management operations.

PROBLEM STATEMENT

Docker images provide a standardized way to package applications and their dependencies. A Dockerfile can be used to automate the creation of customized Docker images. Container registries allow Docker images to be stored, shared, and retrieved when required. Therefore, create a custom Docker image using a Dockerfile, run and test the image locally, and manage the image using a container registry.

SOFTWARE REQUIRED

- Computer/Laptop
- Windows 10/11 or Ubuntu Linux
- Docker Desktop for Windows or Docker Engine for Ubuntu
- Text editor such as Notepad or Visual Studio Code
- Internet connection
- Docker Hub account or another container registry account
- Command Prompt/PowerShell or Ubuntu Terminal

INPUT

- Dockerfile
- Base Docker image
- Application files/content
- Image name and tag
- Container registry account details
- Registry repository name

OUTPUT

- A custom Docker image successfully created using a Dockerfile.
- The custom image successfully built and tested locally.
- Docker image tagged with the registry repository name.
- Image successfully pushed to a container registry.
- Image successfully pulled from the registry.
- Docker image management operations successfully demonstrated.

Rajarshi Shahu College of Engineering, Tathwade Pune-33

(An Empowered Autonomous Institute Affiliated to Savitribai Phule Pune University
Pune)

THEORY

A Docker image is a lightweight, portable package containing the files, dependencies and configuration required to run an application. A Dockerfile is a text file containing instructions used by Docker to automatically build an image. A container registry is a service used to store and distribute Docker images. Docker Hub is a commonly used public container registry. An image can be tagged with a registry repository name and pushed to the registry so that it can be pulled and used on another system.

4. Part A – Create a Dockerfile

Step 1: Create a Working Directory

Create a directory named docker-custom-image and open a terminal/PowerShell in that directory.

Step 2: Create the Dockerfile

Create a file named exactly Dockerfile without any file extension.

Dockerfile content:

```
FROM nginx:latest
COPY index.html /usr/share/nginx/html/index.html
EXPOSE 80
```

5. Create the Web Page

Create a file named index.html in the same directory as the Dockerfile.

index.html content:

```
<!DOCTYPE html>
<html>
<head>
  <title>My Custom Docker Image</title>
</head>
<body>
  <h1>Hello from My Custom Docker Image!</h1>
  <p>This web page is running inside an Nginx Docker container.</p>
</body>
</html>
```

6. Build the Custom Docker Image

Open PowerShell/Terminal in the directory containing Dockerfile and index.html.

```
docker build -t mynginx:1.0 .
```

The -t option assigns a name and tag to the image. The dot (.) specifies the current directory as the build context.

7. Verify the Docker Image

```
docker images
```

Verify that mynginx with tag 1.0 appears in the list.

8. Run the Custom Image

```
docker run -d --name customnginx -p 8080:80 mynginx:1.0
```

Check the running container:

```
docker ps
```

Open a web browser and enter:

http://localhost:8080

The custom web page should be displayed.

9. Test and Stop the Container

```
docker ps
```

```
docker stop customnginx
```

```
docker ps -a
```

After stopping, the container should show an Exited status.

10. Tag the Image for Docker Hub

Log in to Docker Hub and create a repository, for example: mynginx.

Replace YOUR_USERNAME with your Docker Hub username.

```
docker login
```

```
docker tag mynginx:1.0 YOUR_USERNAME/mynginx:1.0
```

11. Push the Image to Docker Hub

```
docker push YOUR_USERNAME/mynginx:1.0
```

Wait until the image layers are uploaded successfully.

Rajarshi Shahu College of Engineering, Tathwade Pune-33

(An Empowered Autonomous Institute Affiliated to Savitribai Phule Pune University
Pune)

12. Verify the Image in the Container Registry

Open Docker Hub in a web browser and open the repository created for the image.
Verify that the mynginx image and the 1.0 tag are available in the repository.

13. Pull the Image from Docker Hub

`docker pull YOUR_USERNAME/mynginx:1.0`

The image can now be downloaded from the registry on another Docker-enabled system.

14. Run the Registry Image

`docker run -d --name registrynginx -p 8081:80 YOUR_USERNAME/mynginx:1.0`

Open <http://localhost:8081> in a browser to verify that the image works after being pulled from the registry.

15. Observation

Parameter	Expected / Example	Observation
Dockerfile	Created successfully	
Image Name	mynginx	
Image Tag	1.0	
Container Name	customnginx	
Host Port	8080	
Container Port	80	
Registry	Docker Hub	
Repository	YOUR_USERNAME/mynginx	

16. Important Docker Commands

Operation	Command
Build Image	<code>docker build -t mynginx:1.0 .</code>
List Images	<code>docker images</code>
Run Container	<code>docker run -d --name customnginx -p 8080:80 mynginx:1.0</code>
View Containers	<code>docker ps</code>
Stop Container	<code>docker stop customnginx</code>
Login to Docker Hub	<code>docker login</code>
Tag Image	<code>docker tag mynginx:1.0 YOUR_USERNAME/mynginx:1.0</code>
Push Image	<code>docker push YOUR_USERNAME/mynginx:1.0</code>
Pull Image	<code>docker pull YOUR_USERNAME/mynginx:1.0</code>
Remove Image	<code>docker rmi mynginx:1.0</code>

17. Learning Outcome

- Students understand how a Dockerfile is used to create a custom Docker image.
- Students can build, tag and run a custom Docker image.
- Students understand how images are stored and distributed using a container registry.

CONCLUSION

Thus, the process of creating a custom Docker image using a Dockerfile and managing the image in a container registry was successfully demonstrated.